

A standardization proof of Foata compatibility

Research note - October 4, 2026.

Result. For every unit interval parking function α ,

$$\text{spot}_{F(\alpha)} = F(\text{spot}_\alpha).$$

This is the identity stated as Conjecture 6.3 in Celano et al., *ℓ -Interval Parking Functions: Foata invariance, statistic generating functions, and ciphers*, arXiv:2507.07243v2. The argument below proves the identity under that paper's conventions. It does not establish publication priority.

1. Definitions and the short proof

Cars numbered $1, \dots, n$ arrive in that order, with preferences $\alpha = (a_1, \dots, a_n)$. A car takes the first unoccupied spot at or to the right of its preference. Write s_i for the spot taken by car i . The word is a **unit interval parking function** when every car parks and $0 \leq s_i - a_i \leq 1$. The permutation $\text{spot}_\alpha = (s_1, \dots, s_n)$ records spots by arrival order.

The **stable standardization** $\text{std}(w)$ ranks a word's entries by increasing value, breaking equal-value ties from left to right. For instance,

$$\text{std}(3, 1, 1, 4) = (3, 1, 2, 4).$$

Three facts imply the desired identity:

1. $\text{spot}_\alpha = \text{std}(\alpha)$ for every unit interval parking function.
2. $\text{std}(F(w)) = F(\text{std}(w))$ for every word.
3. F preserves unit interval parking functions.

Consequently,

$$\text{spot}_{F(\alpha)} = \text{std}(F(\alpha)) = F(\text{std}(\alpha)) = F(\text{spot}_\alpha).$$

Fact 1 is equivalent to the inversion-set assertion in Lemma 2.14 of Celano et al.; Fact 3 is their Proposition 3.9. Fact 2 is a classical compatibility of Foata's transformation with standardization, discussed in §2.3, especially Definition 2.3.9 and the following paragraph, of Gillespie's 2016 dissertation. For completeness, the following sections prove all three facts directly.

2. The Foata convention

Use the transformation in Definition 3.2 of Celano et al. Equivalently, start with the empty output and insert the input letters from left to right. To insert a new letter x into a nonempty current output u :

- If the last letter of u is at most x , cut immediately after every letter at most x .
- Otherwise, cut immediately after every letter greater than x .
- In each resulting segment, move the last letter to the front. Keep the segments in their original order, and append x .

The last letter of u is always a cut position, so these segments partition the whole output.

For a fixed insertion threshold, call the letters after which cuts are made the *cut class*, and all remaining letters the *other class*. Each segment has the form vc , where v consists of letters in the other class and c is a single letter in the cut class. The rotation $vc \mapsto cv$ preserves the relative order of all letters within each class across the entire output. This observation will be used twice.

3. Parking is standardization in the unit interval case

Let α be unit interval, and take $i < j$.

If $a_i > a_j$, then

$$s_j \leq a_j + 1 \leq a_i \leq s_i.$$

Distinct cars occupy distinct spots, so $s_j < s_i$.

If $a_i \leq a_j$, suppose instead that $s_j < s_i$. Spot s_j was still free when car i arrived: its eventual occupant, car j , arrived later, and parked cars never move. Moreover, $s_j \geq a_j \geq a_i$. Car i therefore could not pass that available spot to reach s_i , a contradiction. Hence $s_i < s_j$.

We have proved, for $i < j$,

$$s_i > s_j \iff a_i > a_j.$$

Thus the spot permutation orders the preferences increasingly and resolves ties in arrival order. It is exactly $\text{std}(\alpha)$.

4. Foata commutes with stable standardization

Give each occurrence in a word w its stable rank in $\text{std}(w)$, and carry that distinct label along when the occurrence is moved.

Suppose the next input occurrence has value x and label r . For any previously inserted occurrence with value y and label t ,

$$y \leq x \iff t < r.$$

Indeed, unequal values are ranked by value, and all previously inserted copies of x have smaller labels than this new copy. Therefore the two runs - one on the values and one on their distinct labels - choose the same branch, cut at the same positions, and perform the same rotations. By induction on the number of inserted letters, the final label word is $F(\text{std}(w))$.

It remains to identify these carried labels with the standardization of the final value word. At every insertion, equal-valued occurrences all belong to the same threshold class. The observation in §2 shows that rotations preserve their relative order. A new occurrence is appended after all earlier occurrences of its value. Hence the final order among equal values is the original left-to-right order.

The carried labels are therefore exactly $\text{std}(F(w))$, proving

$$\text{std}(F(w)) = F(\text{std}(w)). \quad \square$$

5. An independent proof of unit interval invariance

First, for a permutation π , Foata's transformation preserves the relative order of every pair of consecutive values $k, k+1$. When the later of these two input values is inserted, it is appended after the earlier one. Every subsequent insertion has threshold either less than k or greater than $k+1$, so the pair always belongs to the same threshold class. Section 2 shows that their order can never change. This is the usual preservation of inverse descents, proved here directly.

Now let α be unit interval, let $\beta = (b_1, \dots, b_n)$ be its increasing rearrangement, and set $\sigma = \text{std}(\alpha) = \text{spot}_\alpha$. The occurrence with rank k has preference b_k and parks in spot k . Therefore

$$b_k \in \{k-1, k\}, \quad b_1 = 1.$$

Whenever $b_k = k-1$, the occurrence of rank $k-1$ must precede the occurrence of rank k in σ : spot $k-1$ must already be occupied for the latter car to pass its preference and park at k .

Set $v = F(\alpha)$. Its sorted preferences are still β , because F only rearranges occurrences. By §4,

$$\tau := \text{std}(v) = F(\sigma).$$

The consecutive-value property just proved says that rank $k-1$ still precedes rank k in τ whenever $b_k = k-1$.

Run the parking procedure on v . Induct on arrival order, claiming that each occurrence parks in the spot given by its stable rank. Suppose the next occurrence has rank k , and all earlier occurrences have parked in their respective rank spots. Spot k is free. If $b_k = k$, the car parks there immediately. If $b_k = k-1$, rank $k-1$ has already arrived, so spot $k-1$ is occupied and the car takes spot k . This establishes the induction.

Thus v is unit interval and

$$\text{spot}_{F(\alpha)} = \tau = F(\text{spot}_\alpha),$$

proving both the required invariance and the conjectured identity. $\square$

6. Example and verification

For $\alpha = (3, 1, 1, 4)$,

$$\begin{aligned}\text{spot}_\alpha &= (3, 1, 2, 4), \\ F(\alpha) &= (1, 3, 1, 4), \\ \text{spot}_{F(\alpha)} &= (1, 3, 2, 4) = F(3, 1, 2, 4).\end{aligned}$$

The code below checks the standardization identity on all 9,840 nonempty words of length at most 8 over a three-letter alphabet; consecutive-value order preservation on all 46,233 nonempty permutations of length at most 8; and all four identities/conditions used in the proof on every unit interval parking function of length at most 8. All checks passed.

Length	Unit interval parking functions checked
1	1
2	3
3	13
4	75
5	541
6	4,683
7	47,293
8	545,835
Total	598,444

These finite checks corroborate the implementation and the intermediate statements; the preceding argument establishes the result for every length. The paper already reports checking the conjecture through length 8, so this is an independent reproduction of that range.

```
from itertools import permutations, product

def foata(word):
    out = []
    for x in word:
        if out:
            cut_low = out[-1] <= x
            new = []
            start = 0
            for i, y in enumerate(out):
                if (y <= x) == cut_low:
                    new.append(y)
                    new.extend(out[start:i])
                    start = i + 1
            assert start == len(out)
            out = new
        out.append(x)
    return tuple(out)

def standardize(word):
    result = [0] * len(word)
    indices = sorted(range(len(word)), key=lambda i: (word[i], i))
    for rank, i in enumerate(indices, 1):
```

```

        result[i] = rank
    return tuple(result)

def park(word):
    taken = set()
    result = []
    n = len(word)
    for a in word:
        s = a
        while s in taken:
            s += 1
        if s > n:
            return None
        taken.add(s)
        result.append(s)
    return tuple(result)

def unit_words(n):
    # Enumerate directly from the parking rule. A prefix is retained exactly
    # when all cars so far have parked with displacement at most one.
    def visit(prefix, occupied):
        if len(prefix) == n:
            yield tuple(prefix)
            return
        for a in range(1, n + 1):
            s = a
            while s <= n and occupied & (1 << (s - 1)):
                s += 1
            if s <= n and s - a <= 1:
                prefix.append(a)
                yield from visit(prefix, occupied | (1 << (s - 1)))
                prefix.pop()

    yield from visit([], 0)

assert foata((3, 1, 5, 3, 1, 2)) == (1, 5, 3, 3, 1, 2)
assert foata((3, 1, 1, 4)) == (1, 3, 1, 4)

word_count = 0
permutation_count = 0
unit_count = 0
expected = (1, 3, 13, 75, 541, 4683, 47293, 545835)

for n in range(1, 9):
    for w in product(range(1, 4), repeat=n):
        assert standardize(foata(w)) == foata(standardize(w)), w
        word_count += 1

    for p in permutations(range(1, n + 1)):
        q = foata(p)
        pos_p = {value: i for i, value in enumerate(p)}
        pos_q = {value: i for i, value in enumerate(q)}
        for k in range(1, n):
            assert (pos_p[k] < pos_p[k + 1]) == (
                pos_q[k] < pos_q[k + 1]
            ), p
        permutation_count += 1

count = 0
for w in unit_words(n):

```

```

spots = park(w)
transformed = foata(w)
new_spots = park(transformed)
assert spots == standardize(w), ("parking/standardization", w)
assert new_spots is not None, ("parking success", w)
assert all(
    0 <= s - a <= 1 for s, a in zip(new_spots, transformed)
), ("unit interval invariance", w)
assert standardize(transformed) == foata(standardize(w)), (
    "standardization compatibility", w
)
assert new_spots == foata(spots), ("conjectured identity", w)
count += 1
assert count == expected[n - 1]
unit_count += count
print(f"n={n}: {count} unit interval parking functions; all checks passed")

print(f"Words: {word_count}; permutations: {permutation_count}; UPFs: {unit_count}")
assert (word_count, permutation_count, unit_count) == (9840, 46233, 598444)

```

References and scope

1. Kyle Celano, Jennifer Elder, Kimberly P. Hadaway, Pamela E. Harris, Jeremy L. Martin, Amanda Priestley, and Gabe Udell. *ℓ -Interval Parking Functions: Foata invariance, statistic generating functions, and ciphers*, arXiv:2507.07243v2. Relevant items: Definition 2.2, Lemma 2.14, Definition 3.2, Proposition 3.9, and Conjecture 6.3.
2. Maria Monks Gillespie. *A combinatorial approach to the q , t -symmetry in Macdonald polynomials*, PhD dissertation, University of California, Berkeley, 2016. Section 2.3, printed pages 10-12, gives the same Foata convention, inverse-descent preservation, and compatibility with stable standardization.

The contribution of this note is the deduction of the stated parking identity from these elementary properties, with all needed steps supplied. The standardization compatibility and inverse-descent preservation are established facts, not claims of new lemmas. The source examined still labels the parking identity a conjecture. A targeted search found no resolution, but this is not a guarantee that the deduction is unpublished or unknown to the authors.